One Canvas: Five Runtimes and Purposeful Mixing

The builder in RosettaDash is written in Angular with NestJS handling the API work. When you export, the same layout can leave as React, Angular, Vue, Svelte, or W3C custom elements. You choose the target at export time; the canvas does not change. The first article introduced that idea. This article shows how it works in the repo: the five Destination Atlas proofs, and the one proof (Svelte) that deliberately embeds another framework in some of its screens.

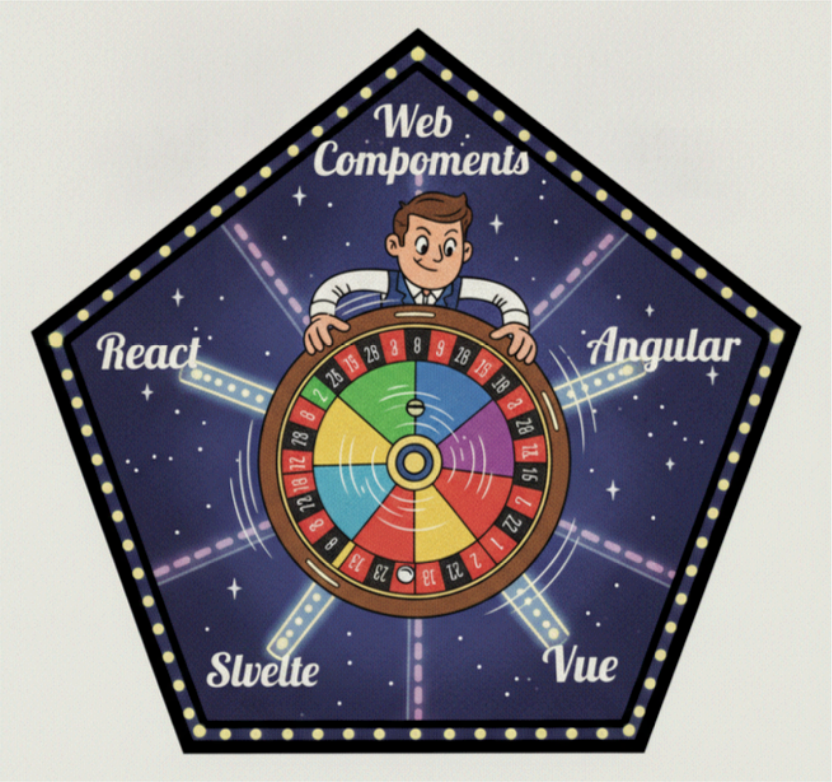

Figure 1: Choose a framework and take a spin.

One foundation, five implementations

Compose on the canvas, fix validation errors, then export. Before any framework-specific files are written, the builder turns your graph into one validated description: which components you placed, how they connect, which env vars and server choices apply, and which UI target you picked. Every exporter reads that same description. Adding another framework does not mean rebuilding the builder — it means adding another exporter that understands the same input.

The default download is standalone source. Drop the zip into a project, set environment variables, and run. You do not need to install `@rosettadash/*` to ship what you just

exported. If you prefer npm instead, install the packages for the framework you already use and import a typed component.

```
npm install @rosettadash/vue
# or: @rosettadash/react |
#     @rosettadash/angular |
#     @rosettadash/svelte |
#     @rosettadash/web-components
```

Everything runs on your machine. The generated files live in your tree.

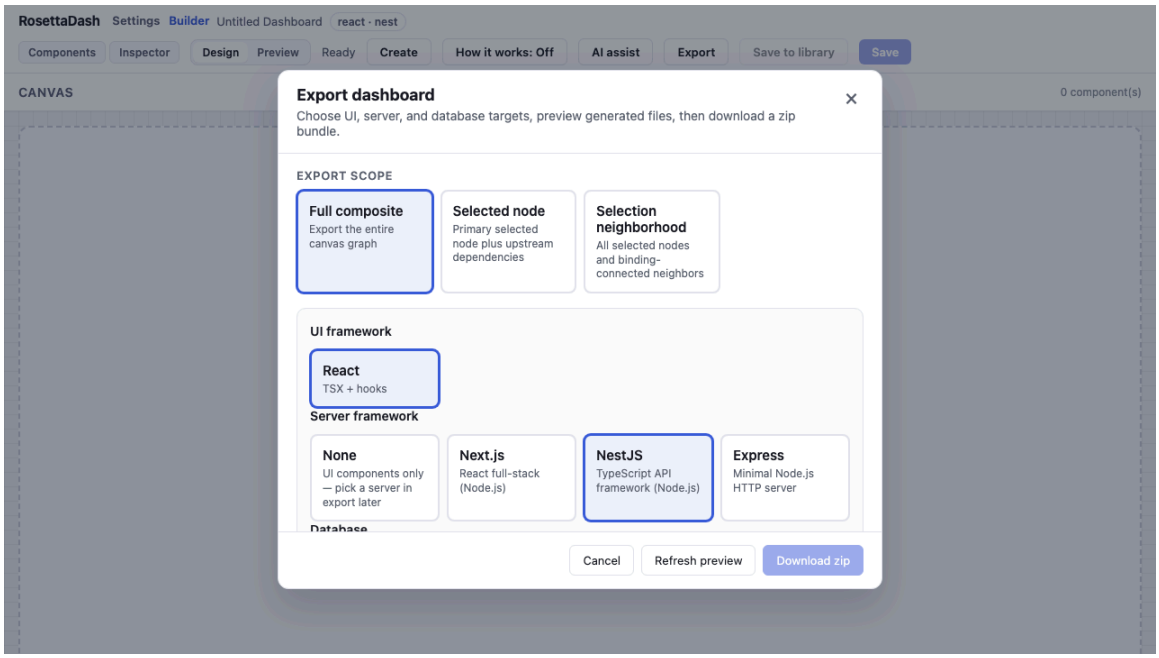

Figure 2: Export target picker — same canvas, choice of framework at export time.

Five proofs, one library

Destination Atlas ships five times — same thirty cities, same screen names, one shared library consumed by every proof app.

Runtime	Command	Port
Web Components	`npm run proof:web-components`	4310
React (reference UX)	`npm run proof:react`	4311
Angular	`npm run proof:angular`	4312
Vue	`npm run proof:vue`	4313
Svelte	`npm run proof:svelte`	4314

Open **About** in any proof. The matrix lists Package, Proof app, and Storybook; the current row, which references the framework being used, is marked “You are here.”

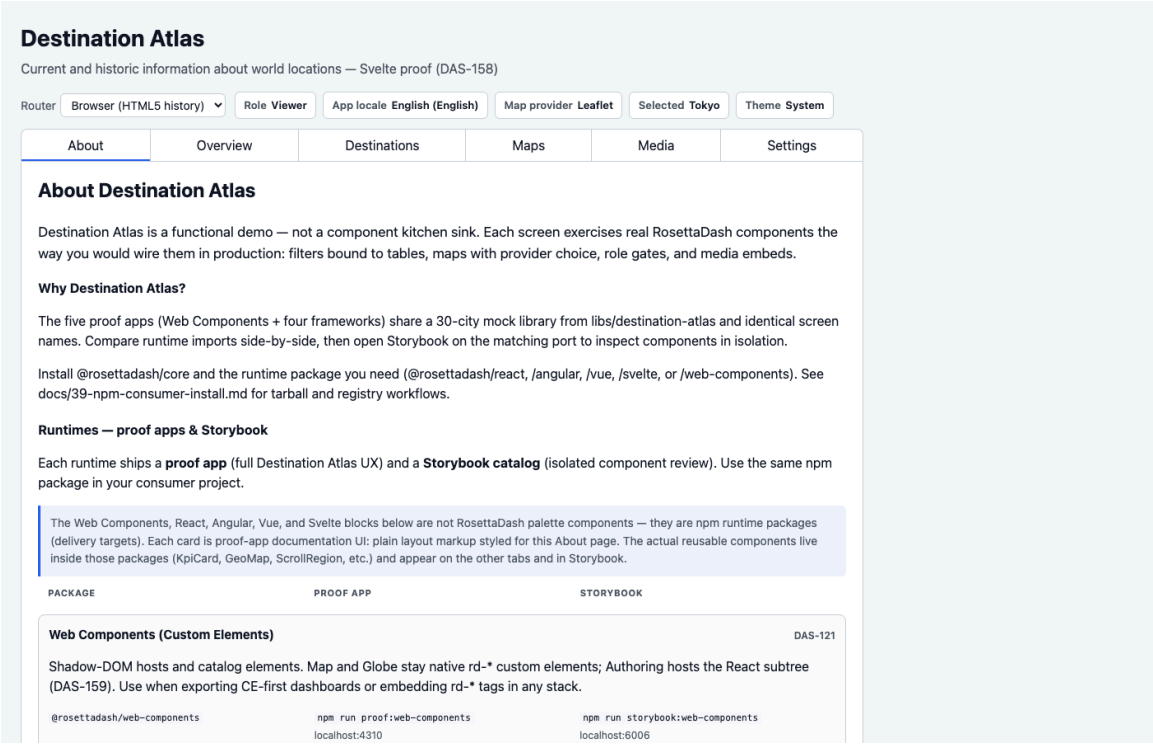

Figure 3: About runtime matrix — Package, Proof app, Storybook. You are here.

Mixing on purpose

As a simple POC, the **Svelte** proof purposefully hosts three framework guests:

- **Globe** — the Vue Three.js wrapper around the geo globe.
- **Media** — the Angular YouTube embed component.
- **Map** — the `<rd-geo-map>` custom element via `svelte:element`.

Authoring is native Svelte — EquirectSphereViewport, FlatVideoViewport, and WasmMedia from `@rosettadash/svelte` (DAS-179).

That is a realistic pattern: reuse what already works, wire it in one app frame, and document the bridge. Storybook showed each piece alone; Destination Atlas shows them together.

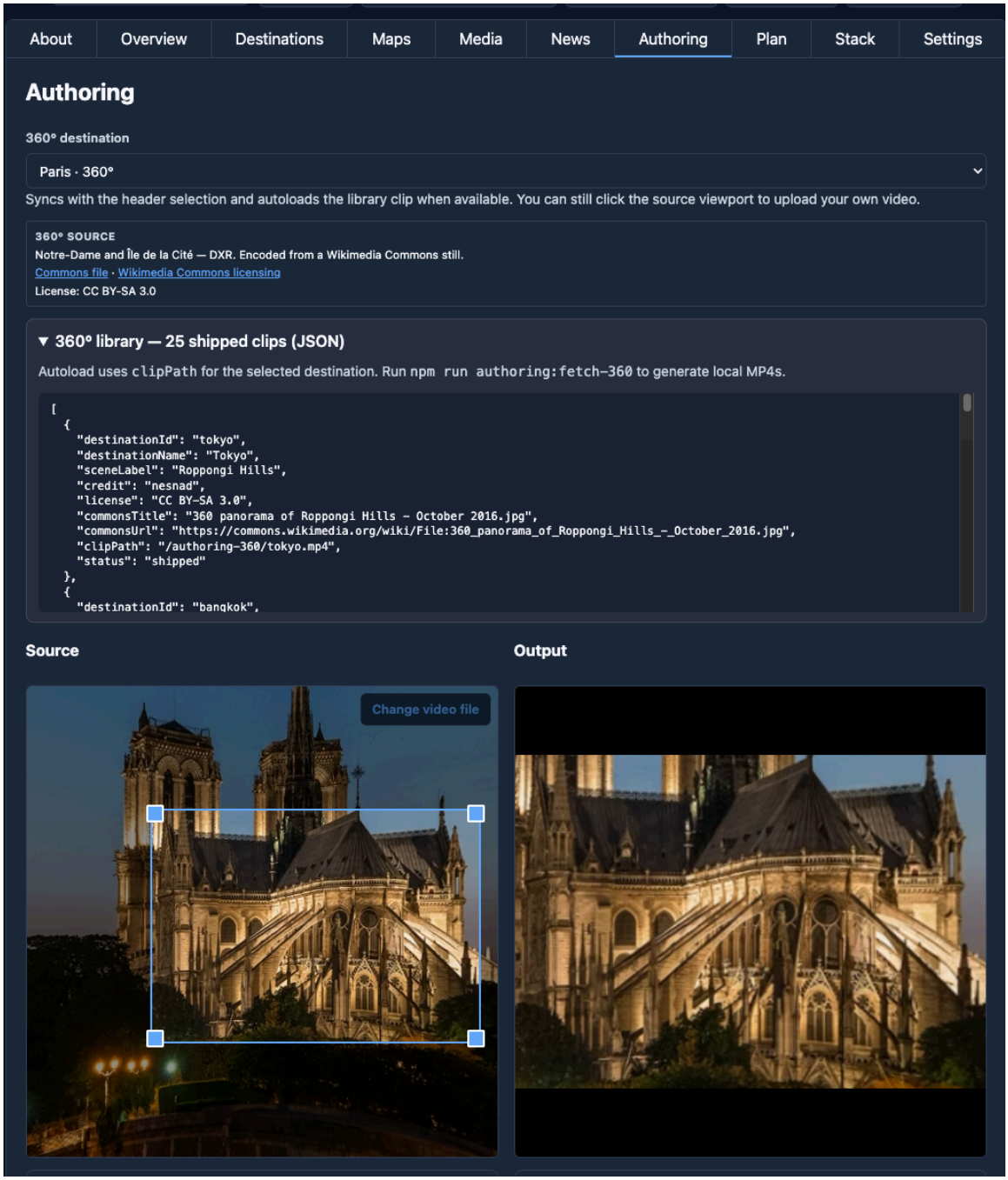

Figure 4: Purposeful mix — Svelte shell with an embedded Vue Globe screen.

The smaller card demos do the same at widget scale: a React host page, a Vue host page, a custom-element 360° tour — each launched with an `npm run demo:*` script from the repo root.

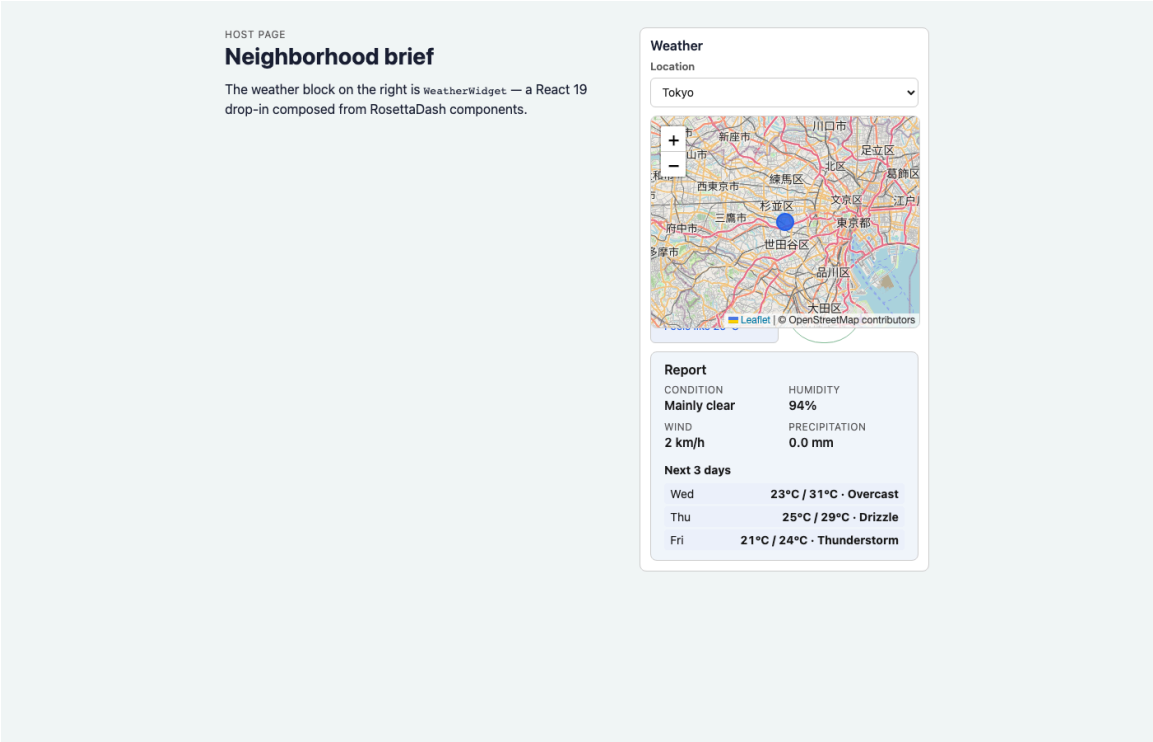

Figure 5: A demo widget on a host page — npm import at card scale.

Close

Clone the repo. Run the proof for the framework you use day to day, or run `npm start` and export a single KPI into a project you already have. Storybook is still there when you want one component in isolation.

The Full List of npm commands for launching apps

Run these from the cloned repo root. Each command starts a dev server; open the URL in your browser.

Builder and API

Command	URL
`npm start`	Builder UI — http://localhost:4200
	NestJS API — http://localhost:3000/api
	Health check — http://localhost:3000/api/health
`npm run start:client`	Builder UI only — http://localhost:4200
`npm run start:server`	NestJS API only — http://localhost:3000/api

`npm start` runs client and server together. Use the split commands when you only need one side.

Backend parity stack (optional)

Proves exported server code against seeded databases in Docker.

Command	Purpose
`npm run parity:verify`	Full check — builder, Docker DBs, generate, servers, matrix
`npm run parity:stack:proof:react`	Live Stack demo — React proof → Next (:53103)
`npm run parity:stack:proof:angular`	Live Stack demo — Angular proof → Nest (:53101)
`npm run parity:stack:storybook:react`	Storybook full-stack orders → Next (:53103)
`npm run parity:stack:storybook:angular`	Storybook full-stack orders → Nest (:53101)
`npm run parity:generate:live`	Regenerate `.parity/servers/` only (builder API up)
`npm run parity:down`	Stop parity containers

Low-level steps (`parity:db:up`, `parity:generate`, `parity:servers:up`) still exist when you need them individually. Live demos use the `:stack:*` scripts so you do not juggle terminals.

Live full-stack demos: React proof Stack → Next; Angular proof Stack → Nest; Storybook **Full-stack orders (live API)**. Same `orders` seed data in every database; generated servers on ports 53101–53104.

Destination Atlas proofs

Same product, five runtimes (ports 4310–4314):

Command	URL
`npm run proof:web-components`	http://localhost:4310
`npm run proof:react`	http://localhost:4311
`npm run proof:angular`	http://localhost:4312
`npm run proof:vue`	http://localhost:4313
`npm run proof:svelte`	http://localhost:4314

Storybook catalogs

One catalog per runtime (ports 6006–6010). All share the same sidebar; **Getting Started → Start here** is the landing story.

Command	URL
`npm run storybook:web-components`	http://localhost:6006
`npm run storybook:react`	http://localhost:6007
`npm run storybook:vue`	http://localhost:6008
`npm run storybook:angular`	http://localhost:6009
`npm run storybook:svelte`	http://localhost:6010
`npm run storybook:all`	All five URLs above (parallel)

Card demos

Standalone widget demos — eleven card-scale hosts (ports 4320–4331):

Command	Runtime	URL
`npm run demo:weather`	React	http://localhost:4320
`npm run demo:news`	React	http://localhost:4321
`npm run demo:sensor`	React	http://localhost:4322
`npm run demo:stock`	Angular	http://localhost:4323
`npm run demo:sports`	Angular	http://localhost:4324
`npm run demo:crypto`	Vue	http://localhost:4326
`npm run demo:flight`	Vue	http://localhost:4327
`npm run demo:media`	Svelte	http://localhost:4328
`npm run demo:transit`	Svelte	http://localhost:4329
`npm run demo:tour`	Web Components	http://localhost:4330
`npm run demo:listing`	Web Components	http://localhost:4331